

SQL Window Functions — Study Notes (Chinook Practice)

1. What Window Functions Are

A **regular aggregate function** (SUM, AVG, COUNT used with GROUP BY) collapses many rows into **one row per group** — individual rows are lost.

A **window function** computes something across a set of related rows **without collapsing them**. Every original row stays in the output, with an extra calculated column added alongside it.

```
SELECT
    name,
    album_id,
    unit_price,
    AVG(unit_price) OVER (PARTITION BY album_id) AS avg_album_price
FROM track;
```

Returns **every track row**, plus the average price of the album it belongs to — computed without merging rows.

Key idea: Window functions “look across a window of rows” for each row’s calculation, but never reduce the row count.

2. Core Terms

Term	Meaning
Window function	A function that operates on a set of rows related to the current row, without collapsing them (ROW_NUMBER(), RANK(), SUM() OVER(...), etc.)
OVER()	The clause that turns a function into a window function. If you see OVER(...), it’s a window function, not a plain aggregate.
Window / Partition	The group of rows a function looks at <i>for each row</i> , defined by PARTITION BY. Like GROUP BY, but doesn’t merge rows.
Window ORDER BY	The ORDER BY <i>inside</i> OVER(...). Controls the order rows are processed in within the window — different from the outer query’s ORDER BY.
Frame	A fine-grained rule for <i>which rows within the partition</i> to include, relative to the current row (ROWS BETWEEN..., RANGE BETWEEN...).

Common Confusion: PARTITION BY vs GROUP BY

Q: Does PARTITION BY reduce the number of rows like GROUP BY does? **Clarification:** No. GROUP BY merges rows into one row per group. PARTITION BY keeps every row — it just tells each row “here are your groupmates” for the calculation. Confirmed with: a table of tracks partitioned by album_id using COUNT(*) OVER(...) still returns the same number of rows as the original table.

Common Confusion: Two different ORDER BYs

Q: If I want the *final result* sorted by trackid, do I add ORDER BY trackid after FROM? **Clarification:** Yes — and this is a **completely separate ORDER BY** from the one inside OVER(...): - ORDER BY **inside OVER()** → controls how the window function computes its result (e.g., ranking order). - ORDER BY **at the end of the query** → just controls the order of the final displayed rows. It has **zero effect** on the window function’s computed values.

```
SELECT name, albumid, unitprice,
       RANK() OVER (PARTITION BY albumid ORDER BY unitprice DESC) AS
price_rank
FROM track
ORDER BY trackid;  -- unrelated to price_rank's calculation, just
sorts output
```

3. Types of Window Functions

3.1 Ranking Functions

(require ORDER BY inside OVER to mean anything)

Function	Behavior on ties
ROW_NUMBER()	Unique sequential number (1,2,3,4...) — ties still get different numbers
RANK()	Same rank for ties, but skips subsequent numbers (two rows tie for 1 → next is 3)
DENSE_RANK()	Same rank for ties, no skipping (two rows tie for 1 → next is 2)

Comparison Table: RANK vs DENSE_RANK vs ROW_NUMBER

Row	Value	ROW_NUMBER()	RANK()	DENSE_RANK()
A	100	1	1	1
B	100	2	1	1
C	100	3	1	1
D	100	4	1	1
E	100	5	1	1
F	90	6	6	2

Common Confusion: Rank skipping

Q: If 5 rows tie for 1st, what rank does the 6th (next distinct) row get under RANK() vs DENSE_RANK()? **Clarification (confirmed correctly):** RANK() → 6 (skips the 5 tied ranks). DENSE_RANK() → 2

(moves to the very next consecutive number).

3.2 Offset / Positional Functions

Function	Purpose
LAG(column, n, default)	Value of column from n rows before the current row
LEAD(column, n, default)	Value of column from n rows after the current row
FIRST_VALUE(column)	Value of column at the first row of the frame
LAST_VALUE(column)	Value of column at the last row of the frame (frame-sensitive — see §5)

LAG() full syntax:

```
LAG(column_name, offset, default_value) OVER (  
    PARTITION BY partition_column  
    ORDER BY order_column  
)
```

- offset (optional, default 1) — how many rows back to look.
- default_value (optional) — value returned instead of NULL when there's no earlier row.
- ORDER BY inside OVER is **essential** — “previous row” is meaningless without a defined order.

```
SELECT payment_date, amount,  
       LAG(amount, 1) OVER (ORDER BY payment_date) AS prev_amount  
FROM payments;
```

Shows each payment plus the amount of the payment before it.
First row's prev_amount is NULL (no earlier row exists).

LEAD() works identically but looks forward.

3.3 Aggregates Used as Window Functions

SUM(), AVG(), COUNT(), MIN(), MAX() — same functions as always, just paired with OVER(...) instead of GROUP BY.

Common Confusion: Adding ORDER BY changes SUM()'s meaning

Q: Does SUM(unit_price) OVER (PARTITION BY albumid ORDER BY trackid) give the same total for every row in an album, or does it grow? **Clarification (confirmed correctly):** Adding ORDER BY inside OVER() for an aggregate **changes its default frame** to a **running total**: “sum of all rows from the start of the partition up to and including the current row.” Without ORDER BY, SUM() sums the *entire partition* for every row (constant value).

```
-- No ORDER BY inside OVER → same total repeated for every row in  
the album  
SUM(unit_price) OVER (PARTITION BY albumid)  
  
-- ORDER BY inside OVER → running total, grows as trackid increases  
SUM(unit_price) OVER (PARTITION BY albumid ORDER BY trackid)
```

4. Filtering on Window Function Results

Common Confusion: WHERE duration_rank = 1 fails

Q: Why does WHERE duration_rank = 1 throw column "duration_rank" does not exist when duration_rank is defined via ROW_NUMBER() in the same query's SELECT? **Clarification (you reasoned this out correctly):** Postgres's logical execution order is roughly: FROM → WHERE → GROUP BY → window functions → SELECT → ORDER BY WHERE runs **before** the SELECT list (and window functions) are evaluated — so duration_rank doesn't exist yet at that point.

Fix — wrap it in a subquery or CTE, then filter in the outer layer:

```
-- Subquery version
SELECT album_id, name, milliseconds
FROM (
    SELECT album_id, name, milliseconds,
           ROW_NUMBER() OVER (PARTITION BY album_id ORDER BY
milliseconds DESC) AS duration_rank
    FROM track
) ranked                                -- ▲ subquery in FROM needs an
alias

WHERE duration_rank = 1;

-- CTE version (cleaner)
WITH ranked AS (
    SELECT album_id, name, milliseconds,
           ROW_NUMBER() OVER (PARTITION BY album_id ORDER BY
milliseconds DESC) AS duration_rank
    FROM track
)
SELECT album_id, name, milliseconds
FROM ranked
WHERE duration_rank = 1;
```

Finds the single longest track per album (the track that is duration_rank = 1).

Common Confusion: Can HAVING be used instead of WHERE?

Q: Could HAVING duration_rank = 1 work directly, without a subquery? **Clarification:** No. HAVING filters **aggregated groups** (from GROUP BY) and can only reference aggregate functions or GROUP BY columns — never a window function's output. This holds even though HAVING *can* technically be used without an explicit GROUP BY (Postgres then treats the whole table as one group, e.g. SELECT COUNT(*) FROM track HAVING COUNT(*) > 100;). The real reason HAVING doesn't work here is that **window functions are not aggregates in the GROUP BY sense** — they operate per-row, not per-group.

QUALIFY — not available in Postgres

Some databases (Snowflake, BigQuery) let you filter window function results directly with QUALIFY, skipping the subquery:

```
-- Snowflake/BigQuery only – NOT valid in Postgres
SELECT album_id, name, milliseconds,
       ROW_NUMBER() OVER (PARTITION BY album_id ORDER BY
milliseconds DESC) AS duration_rank
FROM track
QUALIFY duration_rank = 1;
```

Postgres has no QUALIFY — subquery/CTE is the only option.

5. Frame Clauses (ROWS vs RANGE)

A frame is written as:

```
{ROWS | RANGE} BETWEEN <start> AND <end>
```

Frame boundary types

Boundary	Meaning
UNBOUNDED PRECEDING	The very first row of the partition
N PRECEDING	N rows/values before the current row
CURRENT ROW	The current row itself
N FOLLOWING	N rows/values after the current row
UNBOUNDED FOLLOWING	The very last row of the partition

Rule: <start> must come at or before <end> in this ordering — CURRENT ROW AND UNBOUNDED PRECEDING is invalid (end can't come before start).

```
SUM(milliseconds) OVER (  
  PARTITION BY album_id  
  ORDER BY milliseconds ASC  
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
)
```

Reads as: “for the current row, include everything from the first row of the partition through the current row, counted by physical position.” This is the definition of a running total — the frame grows by one row each time the current row advances.

Common frame patterns

Frame expression	Meaning	Typical use
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	Start of partition → current row	Running total
ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING	Current row → end of partition	“Remaining total from here on”
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	Entire partition, every row	Constant value per partition — same as no ORDER BY at all
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW	Current row + 2 rows before it (3 rows total)	Moving average / moving sum (“last 3 rows”)
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING	One row before, current, one row after (3 rows, centered)	Centered moving average
ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING	Current row + next 2 rows	Forward-looking sum

RANGE uses identical syntax — the difference is purely in what N PRECEDING/N FOLLOWING count by (see below). In practice RANGE with a numeric offset is less common than ROWS, since it depends on the ORDER BY column's actual value gaps rather than row counts; RANGE is mostly used with UNBOUNDED PRECEDING/CURRENT ROW/UNBOUNDED FOLLOWING.

ROWS vs RANGE — what they count by

- **ROWS** — counts by **physical row position**. “2 rows before me” = literally the 2 preceding rows, regardless of their values.
- **RANGE** — counts by **logical value**, based on the ORDER BY column. Rows sharing the **same ORDER BY value** as the current row form one “peer group” and are included/excluded **together**, never split mid-tie.

This only matters when the ORDER BY column has **ties**. With no ties, ROWS and RANGE produce identical results.

Aspect	ROWS	RANGE
Basis	Physical row position	Logical value (based on ORDER BY column)
Ties	Each row counted separately	Rows with the same ORDER BY value form one peer group — all in or all out
Use case	Precise “N rows before/after” (e.g. moving average of last 3 rows)	“All rows with equal value so far” (e.g. running total where tied values count together)

Example (running total with ties): three tracks tied at unit_price = 0.99:

```
SELECT name, unit_price,
       SUM(unit_price) OVER (ORDER BY unit_price
                             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
running_rows,
       SUM(unit_price) OVER (ORDER BY unit_price
                             RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
running_range
FROM track
LIMIT 5;
```

- running_rows (ROWS) — climbs **one row at a time**, even among tied rows; each tied row shows a slightly different running total.
- running_range (RANGE) — treats all rows tied at 0.99 as **one peer group**; every row in that tied group shows the **same** running total (the sum through the whole tied group at once), not a partial mid-tie sum.

Example — moving average of the last 3 tracks (by trackid) in an album:

```
SELECT trackid, album_id, milliseconds,
       AVG(milliseconds) OVER (
         PARTITION BY album_id
         ORDER BY trackid
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS moving_avg_3
FROM track;
```

For each track, averages its milliseconds with the two preceding tracks in the same album (3-row moving average).

Common Confusion: Default frame differs depending on ORDER BY

Q: Why did `SUM() OVER (... ORDER BY ...)` silently become a running total earlier, when no frame was written at all?
Clarification: Postgres applies a **default frame** whenever `ORDER BY` is present but no explicit frame is given — and that default is RANGE-based, not ROWS-based (a subtle point many miss):

Situation	Default frame
Aggregate window function with <code>ORDER BY</code> , no explicit frame	RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
Aggregate window function without <code>ORDER BY</code>	RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING (whole partition)

FIRST_VALUE / LAST_VALUE and the frame trap

```
SELECT album_id, name, milliseconds,
       FIRST_VALUE(name) OVER (PARTITION BY album_id ORDER BY
milliseconds DESC) AS longest_track,
       LAST_VALUE(name) OVER (
           PARTITION BY album_id ORDER BY milliseconds DESC
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS shortest_track
FROM track;
```

⚠ **Classic gotcha:** `LAST_VALUE()` with the *default* frame (RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) only sees rows **up to the current row**, so it usually just returns the current row's own value — not the true last row of the partition. You must explicitly widen the frame to `UNBOUNDED FOLLOWING` to get the actual last value in the partition.

6. Deriving a Boolean Column from a Window Aggregate

You can compare a raw column to a window-function result to produce a TRUE/FALSE column directly — no `CASE WHEN` needed for a simple comparison.

```
SELECT name, album_id, milliseconds, album_avg_ms,
       milliseconds > album_avg_ms AS above_avg
FROM (
    SELECT name, album_id, milliseconds,
           AVG(milliseconds) OVER (PARTITION BY album_id) AS
album_avg_ms
    FROM track
) sub;
```

Inner query computes each track's album average with `AVG() OVER(...)`. Outer query compares `milliseconds > album_avg_ms` — Postgres evaluates this straight to a boolean column.

Common Confusion: Can't compare two columns in the *same* SELECT if one is a window-function alias

Q: Can I write `milliseconds > album_avg_ms AS above_avg` in the **same** `SELECT` that also computes `album_avg_ms` via `AVG() OVER(...)`? **Clarification (you correctly self-diagnosed this):** No — same root problem as the `WHERE duration_rank = 1` case in §4. All columns in one `SELECT` list are conceptually evaluated together; you can't reference another column's *alias* from within

that same list. Wrap the window-function computation in a subquery/CTE, then do the comparison in the **outer** query where the alias already exists as a real column.

Sorting on a derived boolean column

Postgres treats `FALSE < TRUE`, so plain `ORDER BY` behaves predictably:

```
-- FALSE rows first (ascending, default)
ORDER BY above_avg;           -- or: ORDER BY above_avg ASC

-- TRUE rows first, but grouped per album (album_id first, then
above_avg DESC within each album)
ORDER BY album_id, above_avg DESC;
```

Without `album_id` first in the second example, **all** `TRUE` rows across every album would come before **any** `FALSE` row — not per-album grouping. Order columns left-to-right = sort priority.

7. NTILE() — Bucketing Rows

```
SELECT name, album_id, milliseconds,
       NTILE(2) OVER (
         PARTITION BY album_id
         ORDER BY milliseconds DESC
       ) AS duration_half
FROM track;
```

Splits each album's tracks into 2 buckets by duration — longest tracks in bucket 1, shortest in bucket 2.

Common Confusion: Odd number of rows in a partition

Q: If an album has an odd number of tracks, does `NTILE(2)` split them evenly? If not, which bucket gets the extra? **Clarification (confirmed correctly):** `NTILE(n)` distributes any remainder to the **earlier buckets first**, following the window's `ORDER BY`. E.g., 13 tracks into 2 buckets → bucket 1 gets 7, bucket 2 gets 6 — never the reverse.

Other bucketing/percentile functions (same family, not yet practiced with queries):

Function	Purpose
<code>NTILE(n)</code>	Splits partition rows into <code>n</code> roughly equal buckets, numbered 1 to <code>n</code> (e.g. quartiles: <code>NTILE(4)</code>)
<code>PERCENT_RANK()</code>	Relative rank as a fraction between 0 and 1: $(\text{rank} - 1) / (\text{total_rows} - 1)$
<code>CUME_DIST()</code>	Cumulative distribution — fraction of rows with a value $\leq$ the current row's value

```
SELECT name, album_id, unit_price,
       NTILE(4) OVER (ORDER BY unit_price DESC) AS price_quartile
FROM track;
```

Divides all tracks into 4 price-based buckets — useful for “top 25%” style interview questions.

8. Combining Multiple Window Functions in One Query

Different window functions (even with different `PARTITION BY/ORDER BY` clauses) can coexist in the same `SELECT` without interfering with each other — each `OVER(...)` is evaluated independently.

```
SELECT name, album_id, milliseconds,
       RANK() OVER (
         PARTITION BY album_id
         ORDER BY milliseconds DESC
       ) AS duration_rank,
       ROUND(
         milliseconds::numeric / SUM(milliseconds) OVER (PARTITION BY album_id) * 100,
         2
       ) AS pct_of_album_total
FROM track;
```

`duration_rank` ranks each track within its album by length. `pct_of_album_total` independently computes what percentage of the album's total runtime this one track represents. Both window functions run side by side with no conflict.

Common Confusion: Integer division truncates before `ROUND()` ever runs

Q: Why did `milliseconds / SUM(milliseconds) OVER (...) * 100` return mostly `0.00`? **Clarification:** `milliseconds` and the `SUM()` result are both integer types in Postgres. `int / int` performs **integer division**, truncating to a whole number (0 or 1) **before** `* 100` and `ROUND()` are ever applied — so almost every row silently truncates to 0. **Fix:** cast one operand to `numeric` (or `float`) to force proper decimal division:

```
milliseconds::numeric / SUM(milliseconds) OVER (PARTITION BY album_id) * 100
```

This is a classic interview trap any time you divide two integer columns and expect a decimal/percentage result.

9. Window Functions vs GROUP BY — Summary Comparison

Aspect	GROUP BY (regular aggregation)	Window Function (OVER)
Row count in output	Reduced — one row per group	Unchanged — every original row kept
Can mix detail + aggregate columns?	No — non-aggregated columns must be in <code>GROUP BY</code>	Yes — raw row data and aggregate/ranking value live side by side
Filter on result	HAVING	Must wrap in subquery/CTE and use <code>WHERE</code> outside
	“One number per	“Compare each row to its group”

Typical use	category"	(e.g. rank, running total, previous value)
-------------	-----------	--

10. Quick Recap

- A **window function** keeps every row; a **GROUP BY aggregate** collapses rows. `OVER()` is what makes a function a window function.
 - **PARTITION BY** = grouping for calculation purposes only (no row reduction). **ORDER BY inside `OVER()`** = order used *by the calculation*, completely separate from the query's outer `ORDER BY`.
 - **ROW_NUMBER()** always gives unique numbers; **RANK()** skips after ties; **DENSE_RANK()** never skips.
 - **LAG()** / **LEAD()** need `ORDER BY` inside `OVER()` to mean "previous/next row" — always specify it.
 - Adding `ORDER BY` inside `OVER()` to a `SUM()`/`AVG()`/etc. silently turns it into a **running calculation** (default frame becomes `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`).
 - You **cannot** filter or reference a window function's alias again using `WHERE`, `HAVING`, or another expression in the **same** `SELECT` list — always wrap in a **subquery or CTE** and do the filter/comparison in the outer query. Postgres has **no `QUALIFY`**.
 - **Frames** (`ROWS/RANGE BETWEEN <start> AND <end>`) fine-tune exactly which rows within a partition are included — critical for moving averages and for `FIRST_VALUE/LAST_VALUE` correctness. Boundaries: `UNBOUNDED PRECEDING`, `N PRECEDING`, `CURRENT ROW`, `N FOLLOWING`, `UNBOUNDED FOLLOWING` — and `<start>` must come at or before `<end>`.
 - **ROWS counts physical rows; RANGE counts logical values** and keeps tied `ORDER BY` values together as one peer group. They behave identically when there are no ties. The **default frame is RANGE-based**, not ROWS-based.
 - A raw-column-vs-window-aggregate comparison (e.g. `milliseconds > album_avg_ms`) evaluates straight to a boolean column — no `CASE WHEN` needed. Postgres sorts `FALSE < TRUE`, so `ORDER BY` on a boolean column behaves predictably.
 - **NTILE(n)** buckets partition rows into `n` groups; any remainder rows go to the **earlier** buckets first.
 - Multiple window functions can coexist in one `SELECT` — each `OVER(...)` is independent.
 - Dividing two **integer** columns truncates before `ROUND()`/multiplication ever apply — cast one side with `::numeric` whenever you expect a decimal or percentage result.
 - Beyond what we practiced: `PERCENT_RANK()`, `CUME_DIST()` are common in interviews for percentile-style questions.
-

11. Practice Log — Queries Verified This Session

1. **Rank tracks within album by price** (all ties → all rank 1, since Chinook's `unit_price` is uniform at 0.99): `RANK() OVER (PARTITION BY album_id ORDER BY unit_price DESC)`
2. **Rank tracks within album by duration (unique numbering)**: `ROW_NUMBER() OVER (PARTITION BY album_id ORDER BY milliseconds DESC)`
3. **Previous track's duration within album**: `LAG(milliseconds, 1) OVER (PARTITION BY album_id ORDER BY milliseconds DESC)`
4. **Longest track per album** (filtering a window function result): subquery/CTE wrapping `ROW_NUMBER()`, then `WHERE duration_rank = 1`
5. **Boolean flag vs album average**: subquery computing

AVG(milliseconds) OVER (PARTITION BY album_id), outer query
comparing milliseconds > album_avg_ms AS above_avg

6. **2-bucket duration split per album:** NTILE(2) OVER (PARTITION BY album_id ORDER BY milliseconds DESC)
7. **Rank + percent-of-album-total together:** RANK() OVER (...) alongside ROUND(milliseconds::numeric / SUM(milliseconds) OVER (PARTITION BY album_id) * 100, 2)